

Guide TP1 : cadrer une application IA et préparer l'environnement

A. Larhlimi

Développement et déploiement d'applications intelligentes
Semaine 1 · TP de 2 h · travail en binôme · 28 septembre 2026

Ce guide reprend l'énoncé distribué aux étudiants, détaille la démarche pas à pas, explique les figures, chaque commande shell et chaque script du kit `sentiment-app`, puis décrit les problèmes rencontrés en séance et les ressources nécessaires. Les groupes sont des binômes : un *membre A* et un *membre B*.

Table des matières

1	Énoncé du TP1 (version distribuée aux étudiants)	2
1.1	Objectifs	2
1.2	Contenu du kit <code>sentiment-app.zip</code>	2
1.3	Partie A : cadrage (50 min, en binôme)	3
1.4	Banque de douze mini-cas supplémentaires	3
1.5	Partie B : environnement et dépôt (70 min)	4
1.6	Livrables et notation	5
2	Démarche pratique pas à pas	5
2.1	A1. Lire l'exemple corrigé (5 min)	5
2.2	A2. Remplir les trois fiches (40 min)	6
2.3	A3. Vérifier les fiches (5 min)	6
2.4	B1. Outils système (10 min, chacun)	6
2.5	B2. Créer le dépôt GitHub (10 min, membre A)	6
2.6	B3. Initialiser le dépôt à partir du kit (15 min, membre A)	6
2.7	B4. Installer l'environnement (15 min, chacun)	7
2.8	B5. Un commit par membre (10 min, chacun)	7
2.9	B6. Vérification finale (10 min, à deux)	7
2.10	Pour l'enseignant : réponses attendues aux trois cas	7
3	Les figures expliquées	8
3.1	Figure 1. Déroulé du TP1	8
3.2	Figure 2. Circuit Git du binôme	9
3.3	Figure 3. L'environnement virtuel	9
3.4	Figure 4. L'arbre de choix appliqué aux trois cas	10
4	Les commandes shell expliquées	10
4.1	Python et environnement virtuel	10
4.2	Git : créer et publier le dépôt (B3)	11
4.3	Git : travailler à deux (B4, B5)	11
4.4	Vérification et dépannage	11
5	Les scripts et fichiers du kit expliqués	12
5.1	Fichiers de configuration	12
5.2	<code>scripts/setup.sh</code> (Linux, macOS, Git Bash)	12
5.3	<code>scripts/setup.ps1</code> (Windows PowerShell)	13
5.4	<code>scripts/check_setup.py</code> : le diagnostic	13

5.5	<code>scripts/new_cadrage.py</code>	13
5.6	Les tests	14
5.7	<code>.github/workflows/ci.yml</code>	14
6	Problèmes rencontrés en séance	15
6.1	Cas 1. Le dépôt qui pèse 300 Mo	15
6.2	Cas 2. <code>ModuleNotFoundError</code> alors que pip a réussi	16
6.3	Cas 3. Python 3.13 et les versions figées	16
6.4	Cas 4. <code>git push</code> refusé : mot de passe, puis 403	16
7	Démarche à suivre pour le test	16
7.1	Étape 0. Ce que donne le kit fraîchement décompressé	17
7.2	Étape 1. Installer l'environnement complet	17
7.3	Étape 2. Lire le diagnostic	17
7.4	Étapes 3 à 5. Tester par partie, puis tout	17
7.5	Étape 6. Refaire le test de l'enseignant	17
7.6	Résumé des trois états	17
8	Ressources nécessaires	18
8.1	Sur chaque poste, à installer avant la séance	18
8.2	Comptes et accès	18
8.3	Fichiers fournis par l'enseignant	18
8.4	Documentation de référence	19

1 Énoncé du TP1 (version distribuée aux étudiants)

TP1 : cadrer une application IA et préparer l'environnement. Semaine 1, durée 2 h, en binôme (deux étudiants). Le dépôt créé aujourd'hui est le point de départ des dix semaines suivantes : rien de ce qui est fait ici n'est jeté.

1.1 Objectifs

À la fin du TP, chaque binôme dispose :

1. de trois fiches de cadrage argumentées, une par mini-cas ;
2. d'un dépôt GitHub privé `sentiment-app`, structuré, où les deux membres ont poussé au moins un commit ;
3. d'un environnement Python reproductible, vérifié par `python scripts/check_setup.py` et par `pytest`.

1.2 Contenu du kit `sentiment-app.zip`

Fichier	Rôle
<code>README.md</code>	Modèle de README du dépôt : installation en trois commandes, structure, tableau du binôme
<code>TP1-ENONCE.md</code>	Énoncé minuté, livrables, grille de notation, pièges
<code>TESTS.md</code>	Démarche à suivre pour le test, résultats attendus à chaque étape
<code>requirements.txt</code>	Dépendances de tout le semestre, versions figées
<code>.gitignore</code> , <code>pytest.ini</code>	<code>.env.example</code> , Configuration à copier telle quelle
<code>docs/cadrage/TEMPLATE.md</code>	Modèle de fiche de cadrage

Fichier	Rôle
docs/cadrage/cas1-pharmacie.md, cas2-reglement.md, cas3-avis-negatifs.md	Les trois fiches à remplir (partie A)
docs/cadrage/EXEMPLE-corrige.md	Exemple rempli (tri de courriels de support) pour calibrer le niveau attendu
docs/cadrage/BANQUE-DE-CAS.md	Douze cas de rechange : entreprise, citoyens, université, État
scripts/setup.sh, scripts/setup.ps1	Mise en place du poste (Linux et macOS, Windows)
scripts/check_setup.py	Diagnostic lisible de l'environnement
scripts/new_cadrage.py	Crée une fiche supplémentaire à partir du modèle
tests/test_environment.py	Version Python, venv actif, paquets, structure, .gitignore (23 tests)
tests/test_cadrage.py	Trois fiches, rubriques remplies, une approche cochée, deux critères cités (12 tests)
tests/test_git_history.py	Les deux auteurs de commits, .venv/ jamais commité (3 tests)
.github/workflows/ci.yml	Intégration continue : <code>pytest</code> à chaque push
data/sample_reviews.csv	Dix avis d'exemple, utiles dès la semaine 2

1.3 Partie A : cadrage (50 min, en binôme)

Pour chaque mini-cas, remplir la fiche correspondante dans `docs/cadrage/` avec la grille du cours : (a) données étiquetées, (b) vérifiabilité, (c) coût par requête, (d) conséquence d'une erreur.

Fiche	Cas	Indices à discuter
cas1-pharmacie.md	Une pharmacie veut détecter les ordonnances incomplètes	Les champs obligatoires sont-ils connus ? Que coûte une ordonnance validée à tort ? Données de santé
cas2-reglement.md	Une école veut répondre aux questions des étudiants sur le règlement intérieur	Les documents existent déjà. La réponse doit-elle être vérifiable ? A-t-on des questions étiquetées ?
cas3-avis-negatifs.md	Un site e-commerce veut prioriser les avis clients négatifs	Des milliers d'avis. Peut-on les étiqueter ? Que coûte un avis mal classé ?

Règles :

- cocher une seule approche (- [x]) dans « Approche retenue » ;
- la justification cite au moins deux critères par leur lettre, par exemple « (a) ... (d) ... » ;
- chaque rubrique obligatoire contient au moins une phrase complète ;
- deux approches peuvent se défendre pour un même cas : c'est la justification qui est notée, pas la case cochée.

Vérification : `pytest tests/test_cadrage.py -q`.

1.4 Banque de douze mini-cas supplémentaires

Ces sujets servent de banque : pour remplacer un des trois cas, pour un quatrième cas facultatif (`python scripts/new_cadrage.py <fiche> "Titre"`), ou pour varier les sujets d'une promotion à l'autre. Mêmes règles, même fiche, mêmes tests. Ils sont aussi livrés dans le kit, dans `docs/cadrage/BANQUE-DE-CAS.md`.

Domaine	Fiche	Cas	Indices à discuter
Entreprise	ent1-factures-doublons	Une PME veut bloquer les factures fournisseurs payées deux fois	Les critères d'un doublon sont-ils connus (numéro, montant, fournisseur, date)? Le contrôleur doit-il pouvoir expliquer le blocage? Que coûte un double paiement?
Entreprise	ent2-ruptures-stock	Une chaîne de supermarchés veut prévoir les ruptures de stock par magasin et par produit	Historique des ventes sur plusieurs années. Des milliers de prévisions par jour. Que coûte une rupture, que coûte un sur-stock?
Entreprise	ent3-reponses-avis	Une marque veut rédiger des brouillons de réponse aux avis clients	Existe-t-il une bonne réponse unique? Qui relit avant publication? Coût par avis, ton de la marque à respecter
Citoyens (santé)	cit1-triage-urgences	Un centre d'appels d'urgence veut classer les appels par gravité	Des protocoles de triage existent déjà. Que coûte un appel grave classé mineur? Données de santé; qui décide en dernier?
Citoyens (service)	cit2-demarches-commune	Une commune veut répondre aux questions sur les démarches (état civil, urbanisme, taxes)	Les guides et formulaires existent. La réponse doit-elle citer le texte officiel? Les questions contiennent-elles des données personnelles?
Citoyens (assistance)	cit3-chutes-domicile	Une association veut détecter les chutes des personnes âgées à domicile	Enregistrements étiquetés chute ou non chute, mais rares. Que coûte une chute manquée, une fausse alerte? Qui appelle avant d'intervenir?
Université	uni1-decrochage	Repérer à mi-semestre les étudiants à risque d'abandon	Historique étiqueté (diplômé, abandon) sur plusieurs promotions. Risque de stigmatisation? Qui contacte l'étudiant, et pour lui proposer quoi?
Université	uni2-variantes-exercices	Produire des variantes d'exercices corrigés pour les séances de TD	Pas de vérité unique. Que coûte un corrigé faux distribué? Relecture avant diffusion, coût par exercice
Université	uni3-tuteur-cours	Un assistant qui répond à partir des polycopiés et des diapositives du cours	Les supports existent. La réponse doit-elle renvoyer à la page du polycopié? A-t-on des questions étiquetées? Risque : une réponse inventée hors du cours
État (prédiction)	etat1-rendements-agricole	Prévoir les rendements céréaliers par province avant la récolte	Rendements passés, météo, images satellites. Quelques prévisions par an. Que coûte une erreur pour la planification des importations?
État (suivi)	etat2-suivi-chantiers	Répondre aux questions sur l'avancement des chantiers publics	Les rapports d'étape existent et sont mis à jour chaque mois. La réponse doit-elle citer le rapport et sa date? Coût par question?
État (suivi)	etat3-alerte-epidemie	Déclencher des alertes à partir des déclarations hebdomadaires des centres de santé	Des seuils sont définis par les protocoles de surveillance. Que coûte une alerte tardive, une fausse alerte? Les épidémiologistes doivent-ils pouvoir vérifier le déclenchement?

1.5 Partie B : environnement et dépôt (70 min)

1. Installer et vérifier Python 3.11, Git et VS Code (chacun).
2. Créer le dépôt GitHub privé **sentiment-app** dans l'organisation de l'école, ajouter l'autre membre du binôme comme collaborateur et l'enseignant en lecture (membre A).

3. Initialiser le dépôt à partir du kit et pousser sur `main` (membre A).
4. Cloner, créer `.venv`, installer `requirements.txt`, générer `requirements.lock`, lancer le diagnostic (chacun).
5. Pousser une modification réelle : sa ligne dans le tableau du binôme, une rubrique de fiche, ou `requirements.lock` (chacun).
6. Vérifier à deux : `python scripts/check_setup.py`, `pytest -q`, CI verte sur GitHub.

1.6 Livrables et notation

Sur la branche `main`, à la fin de la séance :

- les trois fiches `docs/cadrage/cas*.md` remplies ;
- `README.md` avec le tableau du binôme complété ;
- `requirements.lock` généré ;
- structure de dossiers conforme : `data/`, `models/`, `app/`, `ui/`, `tests/`, `scripts/`, `docs/cadrage/` ;
- un commit par membre visible dans `git shortlog -sn` ;
- CI verte sur GitHub.

Critère	Points	Vérification
Fiches de cadrage : approche justifiée par deux critères, rubriques remplies	4	Lecture + <code>pytest tests/test_cadrage.py</code>
Dépôt clonable, <code>pip install -r requirements.txt</code> sans erreur	2	Clone frais par l'enseignant
Structure et <code>.gitignore</code> conformes, <code>.venv/</code> absent de l'historique	2	<code>pytest tests/test_environment.py</code>
Un commit par membre, messages de commit explicites	2	<code>git shortlog -sn</code> , <code>git log -oneline</code>

2 Démarche pratique pas à pas

Le TP se joue en neuf étapes minutées : trois de cadrage (A), six d'outillage (B). Chaque étape a un résultat vérifiable avant de passer à la suivante.

Étape	Durée	Qui	Résultat attendu
A1 Lire l'exemple corrigé	5 min	le binôme	niveau de détail compris
A2 Remplir les trois fiches	40 min	le binôme	trois fiches, une case cochée, deux critères cités
A3 Vérifier les fiches	5 min	le binôme	<code>pytest tests/test_cadrage.py</code> vert
B1 Outils système	10 min	chacun	Python 3.11, Git, VS Code présents
B2 Créer le dépôt GitHub	10 min	membre A	dépôt privé <code>sentiment-app</code> , membre B collaborateur
B3 Initialiser depuis le kit	15 min	membre A	premier commit poussé sur <code>main</code>
B4 Installer l'environnement	15 min	chacun	<code>.venv</code> créé, 0 erreur(s) au diagnostic
B5 Un commit par membre	10 min	chacun	<code>git shortlog -sn</code> liste les deux membres
B6 Vérification finale	10 min	le binôme	<code>pytest -q</code> vert, CI verte

2.1 A1. Lire l'exemple corrigé (5 min)

Ouvrir `docs/cadrage/EXEMPLE-corrige.md` (tri de courriels de support). Trois choses à remarquer : chaque rubrique tient en une à trois phrases ; la justification cite les critères par leur lettre, « (a) deux ans de courriels étiquetés... (c) plusieurs milliers par jour... (d) une erreur se corrige en un clic » ; la rubrique risques dit quoi faire des données personnelles (anonymisation, hébergement interne).

2.2 A2. Remplir les trois fiches (40 min)

Compter 12 min par cas. Un membre tient le clavier, l'autre parcourt l'arbre de choix dans l'ordre du cours et conteste : logique connue et stable ? règles métier ; données étiquetées ? machine learning ; réponse tirée de documents ? RAG ; sinon génératif seul. Les rôles s'inversent à chaque fiche : en binôme, personne ne reste spectateur.

Chaque fiche comporte huit rubriques obligatoires : besoin en une phrase, utilisateur final, approche retenue, justification, données nécessaires et leur origine, métrique de succès et seuil, conséquence d'une erreur et validation humaine, risques éthiques ou de confidentialité. Une neuvième est facultative : approche écartée et pourquoi.

Conseils qui évitent les échecs de tests : écrire la justification en commençant chaque argument par sa lettre entre parenthèses ; ne cocher qu'une case ; remplacer - [] par - [x] sans changer le reste de la ligne.

2.3 A3. Vérifier les fiches (5 min)

```
pytest tests/test_cadrage.py -q
```

Résultat attendu : 12 passed. Tant qu'une rubrique est vide, qu'aucune ou plusieurs approches sont cochées, ou que la justification cite moins de deux lettres, le test correspondant échoue et nomme la fiche fautive.

2.4 B1. Outils système (10 min, chacun)

```
python --version      # 3.11.x attendu, 3.10 toléré
git --version
code --version         # VS Code, facultatif mais recommandé
```

Windows : installer Python depuis python.org en cochant « Add python.exe to PATH » ; si plusieurs versions cohabitent, `py -3.11 -version` désigne la bonne. macOS : `python3 -version`. Une version 3.12 ou 3.13 déclenche un avertissement du diagnostic et peut faire échouer l'installation des paquets figés (section 6.3).

2.5 B2. Créer le dépôt GitHub (10 min, membre A)

1. Sur GitHub, « New repository » : propriétaire = organisation de l'école, nom **sentiment-app**, visibilité **Private**, sans README, sans `.gitignore`, sans licence (le kit fournit tout cela).
2. Settings → Collaborators and teams → Add people : le membre B en **Write**, l'enseignant en **Read**.
3. Copier l'URL HTTPS de clonage, de la forme `https://github.com/<organisation>/sentiment-app.git`.

2.6 B3. Initialiser le dépôt à partir du kit (15 min, membre A)

```
unzip sentiment-app.zip      # Windows : clic droit, Extraire tout
cd sentiment-app
git init
git add -A
git commit -m "Semaine 1 : structure du depot et kit TP1"
git branch -M main
git remote add origin https://github.com/<organisation>/sentiment-app.git
git push -u origin main
```

Si Git refuse le commit faute d'identité, la définir une fois pour toutes : `git config -global user.name "Prenom Nom"` puis `git config -global user.email "prenom.nom@ecole.ma"`. Chaque membre utilise son propre nom : c'est ce nom que `git shortlog` compte en B6.

Vérifier sur GitHub que `.venv/` n'apparaît pas et que `docs/cadrage/` contient les fichiers `.md`. L'onglet Actions montre un premier passage rouge : fiches vides et un seul auteur, c'est normal, il passe au vert en B6.

2.7 B4. Installer l'environnement (15 min, chacun)

Le membre B clone ; le membre A travaille dans son dossier existant. Chacun lance ensuite le script de mise en place sur son poste.

```
git clone https://github.com/<organisation>/sentiment-app.git
cd sentiment-app
bash scripts/setup.sh
```

```
powershell -ExecutionPolicy Bypass -File scripts\setup.ps1
```

Le script enchaîne quatre étapes : vérifier Python, créer `.venv`, installer `requirements.txt` puis figer l'état exact dans `requirements.lock`, lancer le diagnostic. À la main, c'est équivalent à :

```
python -m venv .venv
source .venv/bin/activate          # Windows : .venv\Scripts\activate
pip install -r requirements.txt
pip freeze > requirements.lock
python scripts/check_setup.py
```

Résultat attendu : 0 erreur(s). Deux avertissements restent normaux à ce stade : fiches incomplètes si la partie A n'est pas finie, et un seul auteur de commits pour l'instant. L'installation dure 5 à 10 min car `requirements.txt` contient déjà tout le semestre (FastAPI, Streamlit, MLflow, DVC, sentence-transformers) : c'est voulu, pour ne plus y revenir.

`requirements.lock` est régénéré sur chaque poste et diffère légèrement d'un système à l'autre : un seul membre le committe en B5, l'autre ne l'ajoute pas.

2.8 B5. Un commit par membre (10 min, chacun)

```
git pull
git add README.md
git commit -m "README : ajout de <nom> au tableau du binome"
git push
```

Si `git push` est refusé (« rejected, non-fast-forward »), l'autre membre a poussé entre-temps : `git pull`, puis, en cas de conflit sur `README.md`, garder les deux lignes du tableau, supprimer les marqueurs «<<<<, =====, >>>>», `git add README.md`, `git commit`, `git push`.

2.9 B6. Vérification finale (10 min, à deux)

```
git pull
python scripts/check_setup.py
pytest -q
```

Résultat attendu : 0 erreur(s), 0 avertissement(s) au diagnostic et 38 passed (23 tests d'environnement, 12 de cadrage, 3 d'historique Git). Pousser une dernière fois et vérifier que la CI est verte dans l'onglet Actions : sur GitHub, le workflow recrée un `venv` et clone tout l'historique, donc les mêmes 38 tests y passent. La démarche complète, avec les trois états intermédiaires, est dans `TESTS.md` (section 7).

2.10 Pour l'enseignant : réponses attendues aux trois cas

Ces réponses sont des attendus raisonnables, pas les seules acceptables.

- **Cas 1, pharmacie : règles métier.** (a) pas de données étiquetées au départ ; (b) la vérification doit être explicable ; (d) une ordonnance validée à tort a une conséquence grave, validation du pharmacien obligatoire. Le ML est acceptable si le binôme précise qu'il faudrait d'abord constituer un jeu étiqueté et garder la validation humaine. Vigilance : données de santé, aucune donnée réelle dans le cours.

- **Cas 2, règlement intérieur : RAG.** (b) la réponse doit citer l'article ; (a) pas de questions étiquetées ; (c) quelques dizaines de questions par jour, le coût par appel est acceptable. Un génératif seul est refusé : il inventerait des articles. Métrique : taux de réponses correctes avec citation exacte sur un jeu de 20 questions.
- **Cas 3, avis négatifs : machine learning.** (a) les avis s'étiquettent facilement, ou l'on part de la note en étoiles ; (c) des milliers d'avis par jour, un classifieur est quasi gratuit ; (d) un avis mal classé est peu grave. Métrique : rappel sur la classe négative, seuil 0,90. C'est le fil rouge du cours, à annoncer en fin de séance.

3 Les figures expliquées

Quatre figures résument le TP : le déroulé minuté, le circuit Git du binôme, l'environnement virtuel et l'arbre de choix appliqué aux trois cas.

3.1 Figure 1. Déroulé du TP1

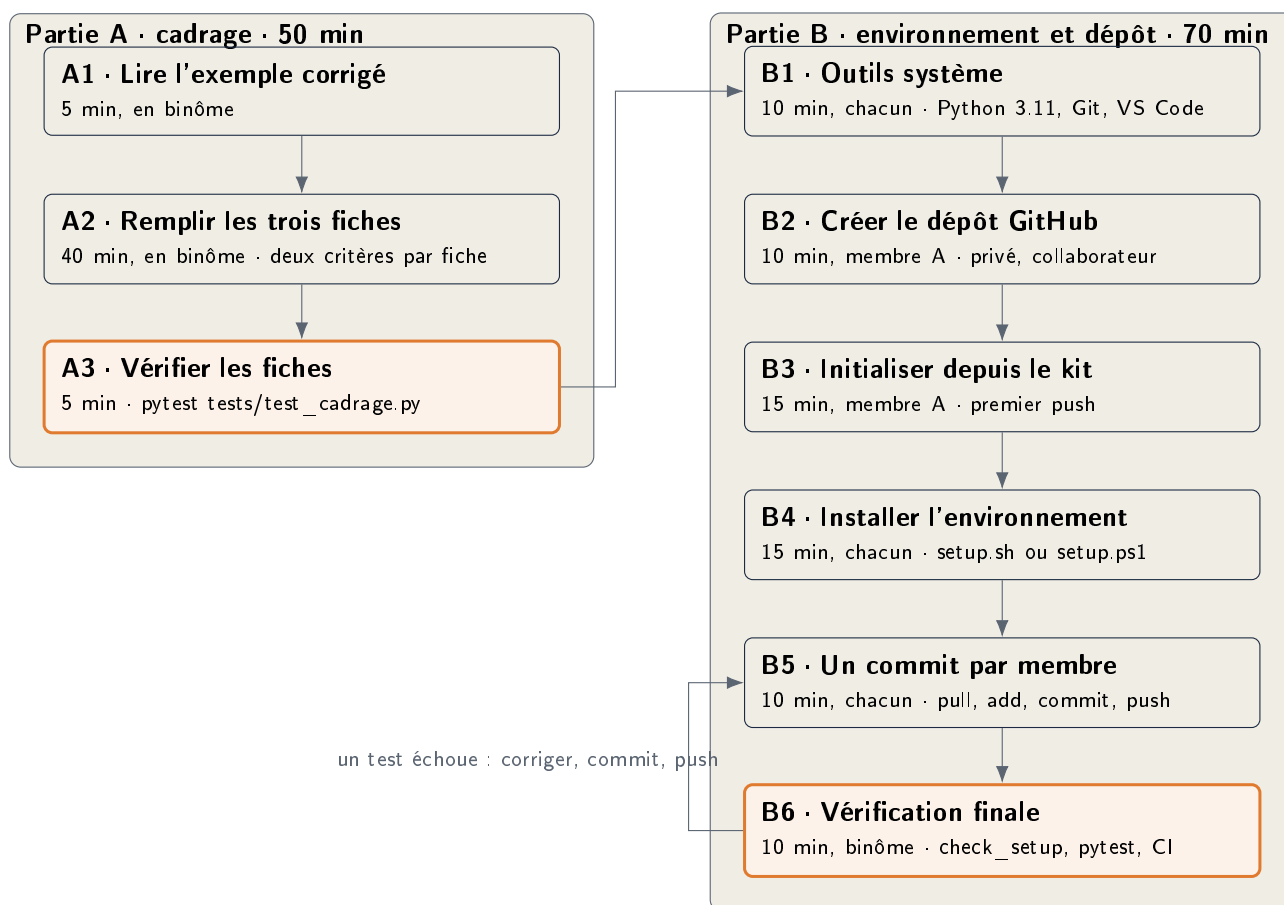


FIGURE 1 – Déroulé du TP1 : neuf étapes, deux parties, une boucle de correction.

Lecture. Les deux colonnes sont les deux parties du TP et se lisent de haut en bas. La partie A tient en trois étapes et se termine par `pytest` sur les fiches ; la flèche qui traverse l'intervalle envoie ensuite le binôme vers B1. Les deux cases colorées, A3 et B6, sont les deux verrous : on ne quitte pas une partie sans un test vert. La boucle à gauche de B6 montre ce qui se passe quand un test échoue : on corrige, on commit, on pousse, et la CI rejoue les tests.

3.2 Figure 2. Circuit Git du binôme

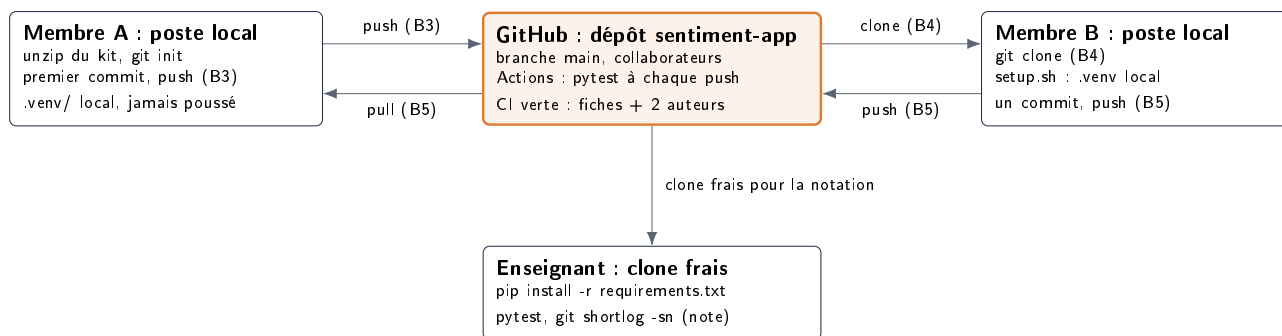


FIGURE 2 – Circuit Git du binôme : quatre acteurs autour du dépôt.

Lecture. GitHub est au centre : rien ne circule directement d’un poste à l’autre. Le membre A pousse le kit une fois (B3) ; le membre B clone (B4), puis chacun pousse son propre commit (B5) et récupère celui de l’autre par `git pull`. Chaque poste garde son `.venv/` local, seuls les fichiers texte partent dans le dépôt. À chaque push, Actions rejoue `pytest` ; l’enseignant note à partir d’un clone frais, exactement comme la CI. Le message à faire passer : si l’enseignant ne peut pas cloner et installer, le travail n’existe pas.

3.3 Figure 3. L’environnement virtuel

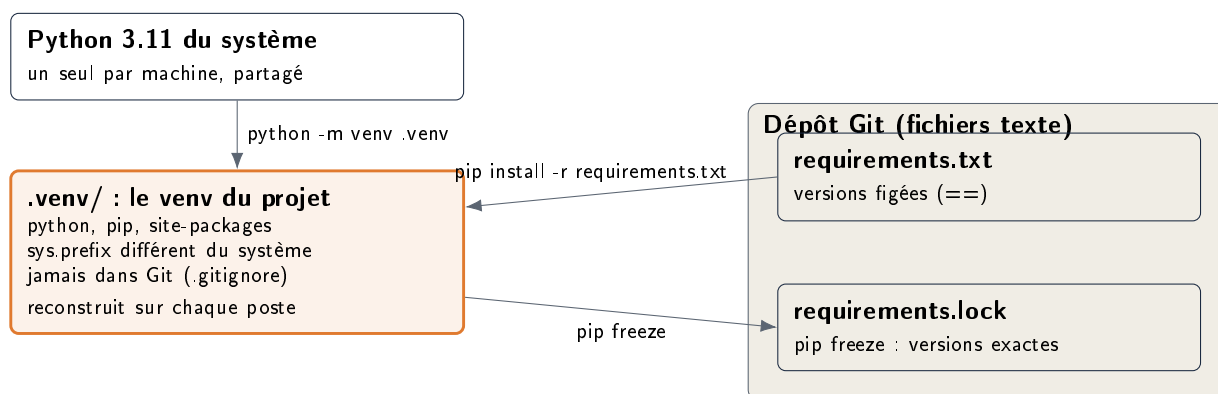


FIGURE 3 – Environnement virtuel : ce qui reste sur le poste, ce qui part dans Git.

Lecture. Le Python du système est unique et partagé ; le venv en est une copie légère, propre au projet, où `pip` installe les paquets du cours. Ce qui voyage dans Git, ce sont deux fichiers texte : `requirements.txt`, écrit par l’enseignant avec des versions figées, et `requirements.lock`, produit par `pip freeze` sur un poste. Le venv ne part jamais dans Git : chaque membre le reconstruit en une commande. `check_setup.py` vérifie qu’il est actif en comparant `sys.prefix` et `sys.base_prefix` : s’ils sont égaux, la commande tourne sur le Python du système, et les paquets installés dans le venv sont invisibles.

3.4 Figure 4. L'arbre de choix appliqué aux trois cas

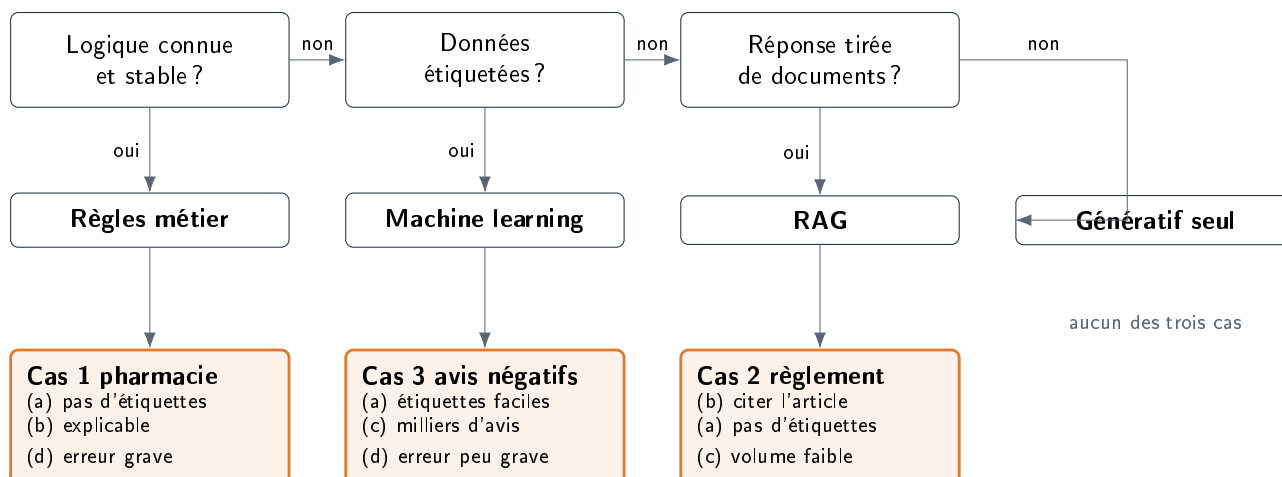


FIGURE 4 – Arbre de choix : trois questions, quatre approches, les trois cas placés.

Lecture. L'arbre se lit de gauche à droite : chaque question reçoit un oui, qui descend vers une approche, ou un non, qui passe à la question suivante. On ne saute jamais au génératif seul : c'est la sortie quand tout le reste a dit non. Les trois cas du TP1 s'arrêtent avant. Les lettres dans les cases sont les critères que la justification de chaque fiche doit citer.

4 Les commandes shell expliquées

Toutes les commandes se lancent depuis la racine du dépôt (le dossier qui contient `README.md`), `venv` activé sauf mention contraire. Le `venv` actif se reconnaît au préfixe `(.venv)` au début de l'invite.

4.1 Python et environnement virtuel

Commande	Ce qu'elle fait	Détail utile
<code>python -version</code>	Affiche la version de l'interpréteur trouvé en premier dans le PATH	Windows : <code>py -3.11 -version</code> cible une version précise; macOS : <code>python3 -version</code>
<code>python -m venv .venv</code>	Crée l'environnement virtuel dans <code>.venv/</code> : une copie de <code>python</code> et <code>pip</code> , un <code>site-packages</code> vide	<code>-m venv</code> exécute le module livré avec Python; le point initial cache le dossier sous Linux et macOS
<code>source .venv/bin/activate</code>	Active le <code>venv</code> dans le terminal courant	PowerShell : <code>.venv\Scripts\Activate.ps1</code> ; cmd : <code>.venv\Scripts\activate.bat</code> ; à refaire dans chaque nouveau terminal
<code>python -m pip install -upgrade pip</code>	Met <code>pip</code> à jour dans le <code>venv</code>	<code>python -m pip</code> garantit que c'est le <code>pip</code> du <code>venv</code> qui tourne
<code>pip install -r requirements.txt</code>	Installe chaque ligne du fichier, versions figées comprises	<code>-r</code> = lire la liste dans un fichier; 5 à 10 min la première fois
<code>pip freeze > requirements.lock</code>	Écrit la liste exacte des paquets installés dans le fichier	<code>></code> redirige la sortie; le <code>lock</code> permet de reproduire l'installation à l'identique
<code>pip install pandas scikit-learn pytest</code>	Installe seulement trois paquets, sans passer par le fichier	Dépannage : rendre <code>pytest</code> disponible vite quand l'installation complète traîne

Commande	Ce qu'elle fait	Détail utile
<code>deactivate</code>	Quitte le venv dans ce terminal	Absente de l'énoncé, mais les étudiants la demandent

4.2 Git : créer et publier le dépôt (B3)

Commande	Ce qu'elle fait	Détail utile
<code>git init</code>	Crée le dépôt local, c'est-à-dire le dossier caché <code>.git/</code>	Un seul membre, une seule fois
<code>git add -A</code>	Met en attente tous les fichiers, sauf ceux exclus par <code>.gitignore</code>	<code>-A</code> = ajouts, modifications et suppressions
<code>git commit -m "..."</code>	Enregistre l'index comme un commit portant ce message	Sans identité configurée, Git refuse
<code>git config -global user.name "..."</code>	Définit l'auteur des commits pour tous les dépôts du poste	<code>git shortlog</code> compte ces noms : un par membre du binôme
<code>git branch -M main</code>	Renomme la branche courante en <code>main</code>	<code>-M</code> force le renommage ; GitHub attend <code>main</code>
<code>git remote add origin <url></code>	Enregistre l'adresse GitHub sous le nom <code>origin</code>	<code>git remote -v</code> l'affiche
<code>git push -u origin main</code>	Envoie <code>main</code> vers <code>origin</code> et lie les deux	<code>-u</code> mémorise le lien : ensuite <code>git push</code> suffit

4.3 Git : travailler à deux (B4, B5)

Commande	Ce qu'elle fait	Détail utile
<code>git clone <url></code>	Télécharge le dépôt avec tout son historique	Le membre B, en B4
<code>git pull</code>	Récupère les commits de l'autre membre et les fusionne	Toujours avant de pousser
<code>git add README.md</code>	Met en attente ce seul fichier	Préférer un fichier nommé <code>-A</code> en B5 : évite d'embarquer un <code>requirements.lock</code> régénéré
<code>git commit -m "..."</code>	Un commit par membre, message explicite	Les messages sont notés (2 pts de la grille)
<code>git push</code>	Envoie les nouveaux commits vers GitHub	Refusé si l'autre membre a poussé entre-temps : <code>git pull</code> , puis <code>git push</code>
<code>git status</code>	Liste les fichiers modifiés, en attente, non suivis	Premier réflexe en cas de doute
<code>git log -oneline</code>	Historique compact, un commit par ligne	Pour relire les messages avant la notation
<code>git shortlog -sn</code>	Nombre de commits par auteur, trié	Ce que lit <code>test_each_member_has_committed</code> (avec <code>-all</code>)
<code>git restore requirements.lock</code>	Annule les modifications locales de ce fichier	Avant <code>git pull</code> si Git refuse à cause du lock régénéré

4.4 Vérification et dépannage

Commande	Ce qu'elle fait	Détail utile
<code>python scripts/check_setup.py</code>	Lance le diagnostic lisible; code de sortie 1 si une erreur bloque	Sans venv actif, la première ligne ERREUR le dit
<code>pytest -q</code>	Lance les 38 tests de <code>tests/</code> en mode silencieux	<code>-q</code> = quiet; <code>-ra</code> (dans <code>pytest.ini</code>) résume échecs et tests sautés
<code>pytest tests/test_cadrage.py -q</code>	Lance un seul fichier de tests	Partie A
<code>.venv/bin/python -m pytest</code>	Lance pytest avec le Python du venv sans l'activer	Windows : <code>.venv\Scripts\python -m pytest</code>
<code>which python</code>	Affiche quel python répond	Windows : <code>where python</code> ; le chemin doit contenir <code>.venv</code>
<code>git rm -r -cached .venv</code>	Retire <code>.venv/</code> du suivi Git sans toucher au dossier	À suivre d'un commit
<code>bash scripts/setup.sh</code>	Exécute le script de mise en place	Linux, macOS, Git Bash
<code>powershell -ExecutionPolicy Bypass -File scripts\setup.ps1</code>	Exécute le script PowerShell en ignorant la politique d'exécution	Évite « running scripts is disabled on this system »
<code>python scripts/new_cadrage.py cas4-mon-cas "Titre"</code>	Crée une fiche à partir du modèle	Facultatif, pour un quatrième cas
<code>unzip sentiment-app.zip</code>	Décompresse le kit dans un dossier <code>sentiment-app</code>	Windows : clic droit, Extraire tout

5 Les scripts et fichiers du kit expliqués

Le kit contient trois familles de fichiers : la configuration du dépôt, les scripts de mise en place et de diagnostic, les tests. Les scripts parlent aux humains (messages lisibles), les tests parlent aux machines (CI, notation); ils vérifient les mêmes choses.

5.1 Fichiers de configuration

README.md. Trois blocs : l'installation en trois commandes, l'arborescence commentée (quel dossier se remplit quelle semaine), le tableau du binôme (membre A, membre B) que chacun complète en B5. C'est le premier fichier que l'enseignant lit sur un clone frais.

requirements.txt. Une ligne par paquet, version figée avec un joker de correctif : `pandas==2.2.*` accepte 2.2.0 ou 2.2.3, jamais 2.3. Le fichier couvre tout le semestre pour ne l'installer qu'une fois; **test_requirements_pins_versions** vérifie que chaque ligne contient `==`.

.gitignore. Quatre groupes de motifs : l'environnement Python (`.venv/`, `__pycache__/`, `*.pyc`), les secrets (`.env`), les artefacts lourds (`models/*`, `data/*`, `mlruns/`, `mlflow.db`) et les fichiers d'éditeur. Les lignes qui commencent par `!` sont des exceptions : `!models/.gitkeep` garde le fichier vide qui force Git à conserver le dossier, `!data/sample_reviews.csv` laisse passer le jeu d'exemple.

.env.example. Modèle des variables secrètes (`API_KEY`, `API_URL`) à copier en `.env`, lui-même ignoré par Git. Le fichier est vide de vrais secrets, on le versionne.

pytest.ini. Deux lignes : `testpaths = tests` et `addopts = -ra` (résumé final des échecs et des tests sautés, avec leur raison).

Les fiches de cadrage. Une rubrique = un titre `####` suivi d'une consigne en citation et d'un espace à remplir. Les tests repèrent les rubriques par leur titre et ignorent les lignes de consigne. **EXEMPLE-corrige.md** a le même format, rempli; **BANQUE-DE-CAS.md** liste les douze cas de rechange.

5.2 scripts/setup.sh (Linux, macOS, Git Bash)

```
set -euo pipefail
cd "$(dirname "$0")/.."
```

```
PY=$(command -v python3.11 || command -v python3 || command -v python)
[ -d .venv ] || "$PY" -m venv .venv
source .venv/bin/activate 2>/dev/null || source .venv/Scripts/activate
```

- `set -euo pipefail` : le script s'arrête à la première commande en échec, à la première variable non définie, et un tube échoue si l'une de ses commandes échoue. Sans cela, un `pip install` raté serait suivi d'un diagnostic trompeur.
- `cd "$(dirname "$0")/.."` : `$0` est le chemin du script, `dirname` son dossier, `..` la racine du dépôt. Le script fonctionne donc quel que soit le dossier d'où on le lance.
- `command -v a || command -v b` : prend le premier interpréteur trouvé, en préférant `python3.11`.
- `[ -d .venv ] || ...` : ne recrée pas le venv s'il existe, le script peut être relancé sans risque.
- L'activation vit dans `bin/` sous Linux et macOS, dans `Scripts/` sous Git Bash : on essaie l'un puis l'autre.
- Ensuite : `pip install -upgrade pip`, `pip install -r requirements.txt`, `pip freeze > requirements.lock`, `python scripts/check_setup.py`.

5.3 scripts/setup.ps1 (Windows PowerShell)

```
$ErrorActionPreference = "Stop"
Set-Location (Join-Path $PSScriptRoot "..")
if (-not (Test-Path ".venv")) { py -3.11 -m venv .venv }
& .\venv\Scripts\Activate.ps1
pip freeze | Out-File -Encoding utf8 requirements.lock
```

Même logique avec les idiomes PowerShell : `$ErrorActionPreference = "Stop"` joue le rôle de `set -e`; `$PSScriptRoot` est le dossier du script; `py -3.11` est le lanceur Windows qui choisit la version; & exécute le script d'activation. Le script se lance avec `-ExecutionPolicy Bypass` parce que la politique par défaut de Windows interdit les scripts non signés.

5.4 scripts/check_setup.py : le diagnostic

Le script imprime une ligne par vérification (OK, ATTENTION, ERREUR), un résumé, et renvoie le code de sortie 1 si une erreur bloque. Les avertissements ne bloquent pas : ils signalent ce qui doit être fini avant la fin du TP.

```
ROOT = Path(__file__).resolve().parents[1]
in_venv = sys.prefix != getattr(sys, "base_prefix", sys.prefix)
MIN_AUTHORS = 2 # un binome : deux auteurs de commits
```

- `ROOT` : le script est dans `scripts/`, `parents[1]` remonte jusqu'à la racine. Tous les chemins sont relatifs à `ROOT`, jamais au dossier courant.
- `check_python` : version via `sys.version_info` (3.11 = OK, 3.10 ou plus = avertissement, en dessous = erreur); venv actif si `sys.prefix` diffère de `sys.base_prefix`. Sur le Python du système, les deux sont égaux : c'est l'erreur la plus fréquente en séance.
- `check_packages` : `importlib.import_module` sur `pandas`, `sklearn`, `pytest`; le dictionnaire fait le lien entre le nom du module (`sklearn`) et le nom `pip` (`scikit-learn`).
- `check_structure` : existence des sept dossiers et des trois fichiers requis; présence de cinq motifs dans `.gitignore`, en avertissement seulement.
- `check_git` : Git dans le `PATH`, `.git/` présent, `git shortlog -sn -all` découpé sur la tabulation puis compté (2 = OK, moins = avertissement), et `git ls-files` qui ne doit pas contenir `.venv/`.
- `check_cadrage` : trois fichiers `docs/cadrage/cas*.md`, et exactement un - [x] dans chacun.

5.5 scripts/new_cadrage.py

Usage : `python scripts/new_cadrage.py cas4-mon-cas "Titre lisible"`. Le script vérifie qu'il reçoit deux arguments, que le nom de fichier ne contient que minuscules, chiffres et tirets

(`re.fullmatch(r"[a-z0-9-]+", slug)`), qu'il n'écrase rien, puis copie `TEMPLATE.md` en remplaçant le titre. Codes de sortie Unix : 0 succès, 1 fichier existant, 2 mauvais usage.

5.6 Les tests

`tests/conftest.py` fournit deux fixtures partagées, `root` et `cadrage_dir`, calculées une fois par session : un test qui déclare un paramètre `root` en reçoit la valeur. C'est la première rencontre des étudiants avec l'injection de dépendances de `pytest`.

Test	Ce qu'il vérifie	Pourquoi
<code>test_python_version</code>	<code>sys.version_info</code> au moins 3.10	Les paquets figés ciblent 3.11
<code>test_virtualenv_active</code>	<code>sys.prefix</code> différent de <code>sys.base_prefix</code>	Force l'habitude d'activer le <code>venv</code>
<code>test_core_package_importable (x3)</code>	<code>pandas</code> , <code>sklearn</code> , <code>pytest</code> s'importent	<code>pip install -r</code> a bien tourné
<code>test_required_directory_exists (x7)</code>	Les sept dossiers de l'arborescence	Structure commune à tous les binômes
<code>test_required_file_exists (x4)</code>	<code>README.md</code> , <code>requirements.txt</code> , <code>.gitignore</code> , <code>.env.example</code>	Rien n'a été supprimé du kit
<code>test_requirements_pins_versions</code>	Chaque ligne utile contient <code>==</code>	Reproductibilité
<code>test_gitignore_contains (x5)</code>	<code>.venv/</code> , <code>__pycache__/</code> , <code>.env</code> , <code>models/</code> , <code>mlruns/</code>	Les motifs qui évitent les dépôts de 300 Mo
<code>test_no_real_env_file_committed</code>	<code>.env</code> et <code>.venv/</code> absents de <code>git ls-files</code>	Sauté hors dépôt Git

Le (xN) vient de `@pytest.mark.parametrize` : une fonction, N cas, N lignes dans le rapport.

`tests/test_cadrage.py` (12 tests) isole le texte d'une rubrique avec une expression régulière :

```
pattern = re.compile(rf"### {re.escape(title)}.*?\n(?:\n|Z)", re.M | re.S)
```

La ligne qui commence par `####` et le titre, puis tout jusqu'au prochain `####` ou la fin du fichier. `re.M` fait agir `^` en début de ligne, `re.S` laisse `.` traverser les sauts de ligne, `re.escape` protège l'apostrophe de « Conséquence d'une erreur ». Les cinq contrôles : trois fiches existent ; les huit rubriques sont présentes dans chacune ; leur corps fait au moins 20 caractères hors consigne ; exactement un - [x] ; au moins deux critères cités dans la justification.

`tests/test_git_history.py` (3 tests) saute tout si Git manque, si `.git/` n'existe pas ou s'il n'y a aucun commit : sur un kit fraîchement décompressé, ces tests apparaissent en `skipped`, pas en échec. Les trois contrôles : au moins un commit ; au moins `MIN_AUTHORS = 2` auteurs distincts, un par membre du binôme ; `.venv/` absent de tout l'historique (`git log -all -name-only`), ce qu'un `git rm -cached` tardif ne suffit donc pas à corriger.

5.7 .github/workflows/ci.yml

```
on: [push, pull_request]
- uses: actions/checkout@v4
  with: { fetch-depth: 0 }
- run: python -m venv .venv && . .venv/bin/activate && pip install pandas scikit-learn pytest
- run: . .venv/bin/activate && pytest -q
```

- `on: [push, pull_request]` : les tests tournent à chaque push et à chaque pull request.
- `fetch-depth: 0` : par défaut Actions clone un seul commit ; avec 0, tout l'historique est récupéré, sinon `git shortlog` ne verrait qu'un auteur et le test échouerait toujours.
- Le workflow crée un `venv` et l'active dans chaque `run`, sinon `test_virtualenv_active` échouerait ; il n'installe que trois paquets pour que la CI dure une minute.

6 Problèmes rencontrés en séance

Quatorze symptômes reviennent d'un binôme à l'autre ; le tableau donne la cause et la correction, puis quatre cas sont racontés en détail.

Symptôme	Cause	Correction
Le dépôt pèse 300 Mo, <code>git push</code> traîne ou est refusé	<code>.venv/</code> commité (<code>.gitignore</code> absent au moment du <code>git add -A</code>)	<code>git rm -r -cached .venv</code> , vérifier <code>.gitignore</code> , commit ; si l'historique est déjà poussé, cas 1
<code>ModuleNotFoundError: No module named 'pandas'</code>	venv non activé, ou <code>pip</code> d'un autre Python que <code>python</code>	activer le venv, <code>which python</code> , <code>python -m pip install -r requirements.txt</code> ; cas 2
<code>test_virtualenv_active</code> échoue alors que tout fonctionne	<code>pytest</code> lancé hors du venv	activer le venv, ou <code>.venv/bin/python -m pytest</code>
<code>No matching distribution found for scikit-learn==1.5.*</code>	Python 3.12 ou 3.13 sans paquets précompilés pour la version figée	installer 3.11 à côté, recréer le venv avec <code>py -3.11</code> ; cas 3
<code>Activate.ps1 cannot be loaded</code>	politique d'exécution PowerShell par défaut	<code>Set-ExecutionPolicy -Scope CurrentUser RemoteSigned</code> , ou lancer avec <code>-ExecutionPolicy Bypass</code>
<code>Support for password authentication was removed</code>	GitHub n'accepte plus le mot de passe du compte	jeton d'accès personnel ou <code>gh auth login</code> ; cas 4
403 Forbidden avec un jeton valide	le membre B n'est pas collaborateur, ou n'a pas accepté l'invitation	Settings, Collaborators; accepter l'invitation reçue par courriel
<code>rejected, non-fast-forward</code>	l'autre membre a poussé entre-temps	<code>git pull</code> , résoudre le conflit éventuel, <code>git push</code>
<code>Your local changes would be overwritten: requirements.lock</code>	<code>setup.sh</code> a régénéré le lock sur ce poste	<code>git restore requirements.lock</code> , puis <code>git pull</code>
<code>test_each_member_has_committed</code> échoue alors que les deux ont poussé	même <code>user.name</code> sur les deux postes, ou commit sur une branche jamais poussée	<code>git config user.name distinct</code> , refaire un commit; <code>git shortlog -sn -all</code>
CI rouge, tests verts en local	fiches vides poussées, ou un seul auteur sur GitHub	lire le journal d'Actions : ce sont les mêmes tests, le nom du test dit quoi corriger
<code>pip install</code> très long ou coupé	sentence-transformers et ses dépendances sont lourds	<code>pip install pandas scikit-learn pytest</code> d'abord, le reste en fin de séance
<code>test_justification_cites_two_criteria</code> échoue	la justification parle de « données étiquetées » sans écrire (a)	citer les lettres de la grille explicitement
Accents cassés dans le terminal Windows	invite <code>cmd</code> en page de code cp1252	<code>chcp 65001</code> , ou utiliser Windows Terminal et PowerShell
<code>python</code> ouvre le Microsoft Store	alias d'exécution d'application Windows	Paramètres, Applications, Alias d'exécution : désactiver <code>python.exe</code> , ou utiliser <code>py</code>

6.1 Cas 1. Le dépôt qui pèse 300 Mo

Situation. Le membre A crée le venv et installe les paquets avant de décompresser le kit dans le même dossier, ou supprime `.gitignore` par mégarde, puis lance `git add -A` et `git commit`. Le push dure plusieurs minutes et GitHub refuse un fichier de plus de 100 Mo (une bibliothèque compilée de `torch`).

Diagnostic. `git status` avant le commit aurait listé des centaines de fichiers sous `.venv/`; après coup, `git ls-files | grep .venv | head` les montre, `git count-objects -vH` donne la taille, et

`check_setup.py` affiche **ERREUR** `.venv/` est suivi par Git. Le test `test_venv_never_committed` reste rouge même après `git rm -r -cached .venv`, parce qu'il lit tout l'historique.

Correction. Au TP1, où l'historique n'a que quelques commits, repartir de zéro : supprimer `.git`, vérifier que `.gitignore` contient `.venv/`, puis `git init`, `git add -A`, `git status` (aucune ligne `.venv/`), `git commit`, `git branch -M main`, `git remote add origin ...`, `git push -force -u origin main`. Le membre B reclone. Leçon : `git status` avant chaque premier commit, et un `.gitignore` est un fichier qu'on ajoute avant les autres.

6.2 Cas 2. ModuleNotFoundError alors que pip a réussi

Situation. Sous Windows, l'étudiante lance `pip install -r requirements.txt` dans un premier terminal, `venv` actif, puis ouvre VS Code, dont le terminal intégré démarre sans le `venv`. `check_setup.py` affiche **ERREUR** aucun environnement virtuel actif et **ERREUR** `pandas` absent. Variante : `pip` répond à un Python 3.13 global, `python` à un 3.11, et les paquets sont installés dans le mauvais.

Diagnostic. `where python` et `where pip` (Linux, macOS : `which`) montrent deux chemins différents; `pip -version` indique pour quel Python il installe; `python -c "import sys; print(sys.prefix)"` doit pointer dans `.venv`.

Correction. Activer le `venv` dans le terminal courant, et dans VS Code choisir l'interpréteur `.venv` (commande « Python : Select Interpreter ») pour que chaque nouveau terminal l'active seul. Réflexe à prendre : écrire `python -m pip install ...` plutôt que `pip install ...`.

6.3 Cas 3. Python 3.13 et les versions figées

Situation. Un membre a installé le Python le plus récent. `check_setup.py` avertit **version 3.13** : le cours cible 3.11, puis `pip install -r requirements.txt` échoue sur `scikit-learn==1.5.*` ou tente une compilation depuis les sources et s'arrête faute de compilateur C.

Diagnostic. `pip download scikit-learn==1.5.* -no-deps` ne trouve pas de roue pour cette version de Python; le message précise **Requires-Python** ou liste les versions disponibles.

Correction. Installer Python 3.11 à côté (les deux cohabitent), supprimer `.venv`, recréer avec `py -3.11 -m venv .venv` (macOS, Linux : `python3.11 -m venv .venv`), relancer `setup`. Ne pas modifier `requirements.txt` : les deux membres doivent avoir les mêmes versions. Leçon : figer les versions des paquets fige aussi la version de Python, c'est le prix de la reproductibilité.

6.4 Cas 4. git push refusé : mot de passe, puis 403

Situation. Au premier `git push -u origin main`, Git demande un identifiant et un mot de passe; le mot de passe du compte est refusé avec **Support for password authentication was removed**. Une fois un jeton en main, le membre B obtient **403 Forbidden** : il n'a pas été ajouté comme collaborateur en B2, ou n'a pas accepté l'invitation.

Diagnostic. Le premier message parle d'authentification (qui êtes-vous?), le second d'autorisation (avez-vous le droit?). `git remote -v` confirme l'URL; sur GitHub, Settings, Collaborators montre qui a accès et les invitations en attente.

Correction. Créer un jeton d'accès personnel (Settings, Developer settings, Personal access tokens, portée `repo`) et le coller à la place du mot de passe; Git Credential Manager le mémorise. Alternative : `gh auth login`. Pour le 403, le membre A ajoute le collaborateur et le membre B accepte l'invitation. Leçon : deux erreurs qui se ressemblent, deux causes différentes; lire le message avant de réessayer.

7 Démarche à suivre pour le test

Cette section reprend `TESTS.md`, livré dans le kit. Les binômes la suivent dans l'ordre; chaque étape donne le résultat attendu, mesuré sur le kit livré.

7.1 Étape 0. Ce que donne le kit fraîchement décompressé

```
python -m venv .venv
source .venv/bin/activate          # Windows : .venv\Scripts\Activate.ps1
pip install pandas scikit-learn pytest
pytest -q
```

Résultat attendu : **3 failed, 31 passed, 4 skipped**. C'est le point de départ du TP : les 3 échecs sont dans `tests/test_cadrage.py` (fiches vides, partie A) ; les 4 tests ignorés attendent un dépôt Git (3 dans `tests/test_git_history.py`, 1 dans `tests/test_environment.py`) ; les 31 verts confirment que la structure, les paquets et le `.gitignore` sont en place.

7.2 Étape 1. Installer l'environnement complet

`bash scripts/setup.sh` ou `powershell -ExecutionPolicy Bypass -File scripts\setup.ps1`. Le script vérifie Python, crée `.venv`, installe `requirements.txt`, écrit `requirements.lock`, puis lance le diagnostic.

7.3 Étape 2. Lire le diagnostic

`python scripts/check_setup.py` imprime cinq blocs (Python, Paquets, Structure, Git, Fiches de cadrage) puis un résumé `N erreur(s), M avertissement(s)`.

Ligne	Sens
OK	rien à faire
ATTENTION	ne bloque pas, mais doit disparaître avant la fin de la séance (fiches vides, un seul auteur de commits)
ERREUR	bloque : venv non activé, paquet absent, dossier manquant, pas de dépôt Git, <code>.venv/</code> suivi par Git

7.4 Étapes 3 à 5. Tester par partie, puis tout

```
pytest tests/test_cadrage.py -q          # attendu : 12 passed
pytest tests/test_environment.py -q      # attendu : 23 passed
pytest tests/test_git_history.py -q      # attendu : 3 passed
pytest -q                                # attendu : 38 passed
```

Tant qu'un seul membre a commité, `test_each_member_has_committed` annonce 1 auteur(s) de commits, 2 attendus (un par membre du binome). Vérifier avec `git shortlog -sn -all` ; si les deux noms sont identiques, chacun corrige son identité puis refait un commit.

7.5 Étape 6. Refaire le test de l'enseignant

La note se joue sur un clone frais : le reproduire avant de partir, dans un autre dossier.

```
cd /tmp
git clone https://github.com/<organisation>/sentiment-app.git
cd sentiment-app
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python scripts/check_setup.py
pytest -q
```

Si cette suite passe sans intervention manuelle, le TP est rendu.

7.6 Résumé des trois états

État	check_setup.py	pytest
Kit décompressé, pas de dépôt Git, fiches vides	1 erreur, 3 avertissements	3 failed, 31 passed, 4 skipped
Dépôt Git initialisé, un seul auteur, fiches vides	0 erreur, 4 avertissements	4 failed, 34 passed
Fin du TP : fiches remplies, deux auteurs	0 erreur, 0 avertissement	38 passed

8 Ressources nécessaires

8.1 Sur chaque poste, à installer avant la séance

Outil	Version	Où le prendre	Remarque
Python	3.11.x (3.10 toléré)	https://www.python.org/downloads/	Windows : cocher « Add python.exe to PATH » ; éviter 3.12 et 3.13
Git	dernière version stable	https://git-scm.com/downloads	Sous Windows, l'installateur fournit Git Bash, qui exécute <code>setup.sh</code>
VS Code + extension Python	dernière version	https://code.visualstudio.com/	Facultatif : édition des fiches, sélection de l'interpréteur <code>.venv</code>
Git Credential Manager	livré avec Git pour Windows	https://github.com/git-ecosystem/git-credential-manager	Mémoire le jeton GitHub après le premier push
GitHub CLI (gh)	facultatif	https://cli.github.com/	<code>gh auth login</code> remplace la saisie manuelle du jeton
Un terminal			PowerShell ou Windows Terminal, Terminal sous macOS, bash sous Linux

Matériel : un portable sous Windows 10 ou 11, macOS ou Linux, sans GPU ; quelques Go d'espace disque libre pour le `venv` complet ; une connexion internet pendant toute la séance.

8.2 Comptes et accès

- Un compte GitHub par étudiant, créé avant la séance, de préférence avec l'adresse de l'école.
- L'organisation GitHub de l'école, avec des dépôts privés : un dépôt `sentiment-app` par binôme, l'enseignant en lecture.
- Un jeton d'accès personnel par étudiant, ou `gh auth login` : <https://docs.github.com/fr/authentication/keeping-your-account-and-data-secure/managing-your-personal-access-tokens>.
- Les droits des membres sur le dépôt : <https://docs.github.com/fr/organizations/managing-user-access-to-your-organizations-repositories>.
- Facultatif : un compte Google pour la variante Colab, sans installation.

8.3 Fichiers fournis par l'enseignant

Fichier	Contenu	Usage
Diapositives de la semaine 1	Cadrage, grille, arbre de choix, cycle de vie, fil rouge, éthique	Cours de 2 h avant le TP
TP1-ENONCE.md	Énoncé minuté, livrables, grille, pièges	À distribuer aux binômes
sentiment-app.zip	Dépôt de départ : fiches, exemple, banque de cas, scripts, 38 tests, CI	Un exemplaire par binôme, décompressé en B3

Fichier	Contenu	Usage
TESTS.md (dans le kit)	Démarche à suivre pour le test, trois états de référence	Pendant le TP et avant de rendre
TP1-colab.ipynb	Le même TP sur Colab, dépôt Git simulé à deux membres	Étudiants sans poste ou sans droits d'installation
Ce guide	Figures, commandes, scripts, problèmes, ressources	Enseignant et tuteurs de TP
Sculley et al. (NeurIPS 2015)	Ce que coûte un système ML une fois en production	Lecture avant la semaine 2 : https://proceedings.neurips.cc/paper/2015/file/86df7dcfd896fc2674f757a2463eba-Paper.pdf

8.4 Documentation de référence

Sujet	Ressource	À lire pour
Environnements virtuels et pip	Tutoriel Python, chapitre 12 : https://docs.python.org/fr/3.11/tutorial/venv.html	B4, figure 3
Fichiers requirements	https://pip.pypa.io/en/stable/reference/requirements-file-format/	Versions figées
Git, chapitres 1 et 2	Pro Git, 2 ^e édition : https://git-scm.com/book/fr/v2	B3, B5, conflits
Modèle .gitignore Python	https://github.com/github/gitignore/blob/main/Python.gitignore	Configuration
Workflows GitHub Actions	https://docs.github.com/fr/actions/writing-workflows	Lecture du journal de CI
pytest	https://docs.pytest.org/en/stable/getting-started.html	A3, B6, les tests du kit
Python dans VS Code	https://code.visualstudio.com/docs/python/environments	Cas 2, choix de l'interpréteur
Données personnelles	CNDP https://www.cndp.ma/ (loi 09-08) et CNIL https://www.cnil.fr/ (RGPD)	Rubrique « Risques éthiques »